

A Blockchain-based E-voting Protocol with Improved Voter Anonymity

Abstract—In this paper, we demonstrate an attack that breaks the anonymity property of the widely-studied e-voting scheme BLOCKVOTES. To address this, we design a novel and secure e-voting system based on ring signatures. Unlike existing ring signature-based e-voting protocols, the online phase (i.e., the actual vote-casting phase) of our scheme is both highly efficient and fully scalable. This efficiency is achieved through a careful integration of public-key signatures and ring signatures across the online and offline phases. We implemented our scheme on a native institutional blockchain testbed and conducted extensive benchmarking experiments. The performance results confirm the practicality of our protocol for real-time deployment.

Index Terms—E-voting; Blockchain; Anonymity.

I. INTRODUCTION

Being the citizens of a democratic country, we are all aware of the role the elections have to play in the formation of a democratic government. Elections are a medium for the people to ensure that their voice and opinion reaches the democratic heads of the country. Therefore the need for conducting the elections in a fair and just manner is vital in ensuring that the election results are accepted by the people and the government alike. In such a case, any anomaly or even a slight doubt in the election results can lead to chaos in the nation.

Traditional methods of conducting elections include pen and paper-based elections. This method has existed for years and is still practiced in many areas. It guarantees many security properties such as *ballot secrecy*, *integrity*, *eligibility*, *fairness*, etc. But this method lacks transparency and verifiability of results. Due to the manual counting of votes, the voters, as well as the candidates, need to trust the tallying authorities. With the growth of technology, the elections have also started moving towards employing electronic means such as DREs (Direct Recording Equipment) or internet voting. This allows the voter to cast a vote, and the final tally will be computed by the electronic voting machine. So, it eliminates the role of tallying authorities in the election process while ensuring ballot secrecy, integrity, eligibility, fairness, etc. But this method has also not been able to provide the transparency the election system requires. Moreover, both of the above-mentioned methods are vulnerable to malicious or mistaken tampering by parties with access to the ballots. Also, due to the need to visit the polling station to cast votes, these do not help solve the issue of decreasing voter turnout in recent years. Also, government-forming elections are not the only type of elections taking place in the world. There are multiple other areas that require elections now and then to make critical decisions. All these use cases highlight the need for such voting systems which can be used easily and reliably by the

voters on their smart devices without the involvement of any third party.

Blockchain technology has seen tremendous growth in application areas in the last decade. This distributed ledger technology opens doors for many applications due to its features such as transparency, verifiability, immutability, etc. which allow different unknown parties to participate without the need for trust among each other. Considering the security properties required by the electronic voting systems and the features of Blockchain technology, it appears as an enticing option to act as a platform for electronic voting as well. In this work, we look at an existing Blockchain-based e-voting system, *Blockvotes*, and perform its security analysis. Furthermore, we design a new electronic voting protocol that mitigates the vulnerabilities of the previous protocol as well as provides new security features essential for an e-voting system.

Contribution of the Paper. In this paper, the contributions are as follows -

- 1) Analysing an existing Blockchain-based e-voting algorithm called '*Blockvotes*' for its security vulnerabilities. This algorithm has been rigorously analyzed against various security properties required by any electronic voting system and for the presence of loopholes in the algorithm's implementation.
- 2) Designing a new e-voting system that mitigates all the security loopholes that were identified in the existing system as well as ensuring other properties that are necessary for an e-voting system.

II. RELATED WORK

Several proposals exist for the implementation of an e-voting system using Blockchain as an underlying technology. Different proposals employ various cryptographic protocols for ensuring the properties required by an election system. These cryptographic protocols include signature schemes [1], zero-knowledge proofs [2], homomorphic encryption schemes [3], etc to name a few. In this literature survey, we highlight some of the blockchain-based e-voting protocols along with their security claims and possible limitations.

The [3] paper presents a new university scale e-voting scheme named *Broncovote*. In this proposal, the authors have used Ethereum as a platform to implement the voting algorithm in the form of smart contracts. Also, they have used Paillier Homomorphic Encryption, which is a partial homomorphic encryption scheme. The voting scheme allows

the creation of different types of elections as well as provides access control for the same. The main parties involved are the administrator, voters, and creator. Admin is responsible for starting the protocol by deploying the contracts necessary for registration as well as ballot creation. Voters are required to register themselves and if found eligible, they can cast votes during later stages of the election. Creators are the voters with ballot creation permission from the administrator. The use of smart contracts with strong cryptographic protocols guarantees the properties of voter privacy, security, integrity, etc. However, the issue lies with the scalability of the protocol with the number of voters.

In the paper [4], Lee *et. al* have proposed an e-voting system using Blockchain as an underlying technology. The proposed method involves four entities: Authentication Organization (AO), Trusted Third Party (TTP), voters, and the Blockchain. The Authentication organization has a list of eligible voters. Since AO is required to authenticate the voters, it may know who voted for whom, and also it may alter the list of voters. So, the TTP is introduced so as to limit the control and access of the AO. The authenticated voters register their public key addresses with the TTP after verifying themselves to the TTP. The voters then use the same public key to cast their votes, which simply means making a transaction to the candidate's address. During the tallying phase, the votes are filtered using the addresses available with the TTP, and eligible votes are counted. Nonetheless, the protocol is secure as long as the AO and the TTP do not collude with each other.

The paper [1] presents another blockchain-based e-voting scheme utilizing Blockchain as a transparent 'Ballot-box'. The authors have introduced a central authority (CA), which provides a certain degree of centralization in order to ensure some access control in the voting protocol. The proposed scheme proceeds in four phases. The protocol employs strong cryptographic protocols such as *Blind Signatures* to maintain voter anonymity. The voters first register themselves to the CA, after which they generate their public/private key pair. The voter then creates a digital commitment on his candidate choice and gets a blind signature of the CA on the same, which serves as an eligibility token for the voter. To guarantee fairness, the voter just publishes his digital commitment during the voting phase, whose opening values are later revealed during the opening phase so that the tallying may be done. The protocol also allows the voters to change their vote by casting an alteration ballot. The protocol is claimed to provide eligibility, privacy, fairness, and verifiability, by the authors. However, the protocol hinges on the correct behavior of the CA.

Another proposal for an e-voting system is given in the paper [5]. Local authorities are responsible for the registration of eligible voters after authentication. The voters then receive the voting credentials which are then used by the voters for logging into the system. A separate blockchain is maintained for each candidate and any vote cast to that candidate is appended to that blockchain. Voters cast their votes by making a transaction to their preferred candidate. This system ensures

the properties of eligibility, anonymity, verifiability, etc. but it is as secure as the voter's device.

In the paper [2], McCorry *et. al* have provided a thorough analysis of three different voting scenarios namely, decentralized boardroom voting, centralized remote voting, and centralized polling station voting, followed by the implementation of a small-scale decentralized boardroom voting protocol, *Open Vote Network* (OV Net) protocol. The aforementioned protocol is a self-tallying two-round protocol. In the first round, the voters register themselves by publishing their voting key (in the form of $g^{votingkey}$) along with a zero-knowledge proof for the knowledge of the voting key. During the second phase, the voters publish their actual votes in a similar format, along with another zero-knowledge proof for the knowledge of the vote. The zero-knowledge proofs provide security against any malicious action by any party. This protocol uses blockchain as a computing platform for performing tally as well as preventing any retrospective modification of the published votes.

The paper [6] proposes a new e-voting protocol that uses Permissioned Blockchain as the underlying technology. The use of Proof-Of-Authority (POA) consensus with permissioned blockchain allows the authorities to maintain access control over various participating entities as well as bounding them to behave honestly. Multiple institutions are assigned the role of election administrators, who are responsible for setting up of elections. Voters first need to authenticate themselves to receive user credentials, followed by automatic wallet creation using zero-knowledge proofs to maintain anonymity. Voters interact with smart contracts to cast their votes and they receive a transaction ID for the same. The tallying of the results is done by the smart contract itself. The voters are also allowed to verify their votes by using their transaction ID.

In the thesis FKE, a blockchain-based e-voting system using the Waves platform has been proposed. The proposed protocol starts with the registration of the voters, where each voter is required to create his/her own Wave's wallet. After wallet creation, the voters need to undergo eligibility verification. This marks the end of registration after which the actual voting process starts. For casting a vote, the voter needs to spend a specific amount of waves. The voters make a payment of some waves to the candidate's Bitcoin/Wave address. The vote transactions are then verified by checking the voter's identification number. The total number of assets transferred to a candidate's address will determine the election outcome.

Another electronic voting system has been proposed in the paper by CK. It uses cryptographic primitives such as blind signatures on the Blockchain platform to guarantee the security properties such as security, anonymity, and transparency.

In HR17, the authors have used Blockchain technology to serve as a database recording system in order to prevent any malicious or mistaken modification by a third party. So, blockchain is used in this system only after the voting phase is over. This system also uses hash functions and digital signatures to ensure the security and reliability of the system. A set of nodes are responsible for adding the voting data to

the blockchain and the system ensures that each node gets a turn of block creation in a conflict-free manner.

In authors of (FLP), have proposed a multi-channel hybrid voting system called 'Crypto-voting' based on blockchain technology. This e-voting system is proposed to simplify the decision-making process of the internal governments of various associations. The voting system employs the use of side chains in a permission blockchain network. Side chains allow the usage of subordinate chains along with the main chain of the network, while permissioned blockchain provides access control in order to maintain security. The e-voting system uses two linked blockchains, one for voting management and the other one for vote counting.

The paper KYC+ devises a new blockchain-based e-voting system using smart contracts. The proposed system employs the Ethereum blockchain as a platform where e-voting smart contracts are deployed in order to develop a secure, transparent system. The public availability of the votes on Blockchain ensures that the system is transparent.

Takabatake *et. al.* in their work TKO proposed an e-voting system using *Zerocoin* and *Bitcoin*. Zerocoin is used in order to provide anonymity to the voters. An administrator is responsible for the verification of voters to form a list of eligible voters. Voters register their Bitcoin addresses and Zerocoin commitments with the administrator. Later, each voter generates a new Bitcoin address and uses it to publish the zero-knowledge proof for the Zerocoin commitment to show his eligibility as a voter. Finally, the voter casts his vote in an encrypted manner to guarantee fairness. Admin decrypts the votes in the tallying phase and is responsible for counting the votes and displaying the results.

Several researchers have directed their research to identify various properties that an election system should have or other important criteria on which an election system can be judged. The properties identified in these surveys act as a benchmark for various electronic voting systems being designed. Some of such works are discussed here to get an insight into the security properties to be considered while designing a new e-voting system.

One such work is [7], which highlights the required properties of a secure e-voting system after analyzing various e-voting systems. The authors have bifurcated these requirements into two categories, Core requirements, and Additional requirements, based on their importance in designing an e-voting system. Privacy, the correctness of results, unreusability, eligibility, verifiability, and robustness have been highlighted as the core properties that should be there in an e-voting system. While other properties, such as fairness, coercion resistance, etc., have been put into the additional requirements category.

Similarly, the paper [8] by Fatih *et al.* provides a comprehensive analysis of many e-voting systems that have used blockchain as their underlying technology.

III. SECURITY GOALS OF AN E-VOTING SYSTEM

Our first job is to list the security properties an e-voting protocol should have; we rely on [9].

- **Anonymity:** The voters remain anonymous. Only the voter herself knows which candidate she voted for. Privacy protection is a crucial attribute of voting systems used in practice and is very important in publicly verifiable voting schemes.
- **End-to-end verifiability:** Ensures that the correctness of the voting process can be checked. It includes:
 - **Cast-as-intended:** A voter can verify that the encrypted vote contains her chosen candidate.
 - **Recorded-as-cast:** A voter can verify that the system recorded her vote correctly.
 - **Tallied-as-recorded:** Any interested party can verify whether the final tally corresponds to the recorded votes.

A voting system satisfying all these properties is said to be *end-to-end verifiable*.

- **Self-tallying:** This ensures that any interested party can compute the tally with available data.
- **Resistance against multiple voting:** This fundamental requirement ensures that each eligible voter can contribute at most one valid vote to the final election tally. The system must reliably prevent or detect any attempts by an individual to cast multiple votes that could improperly affect the outcome.
- **Allowing vote overwriting:** During the voting phase, a voter will be able to overwrite his/her own vote.
- **Coercion resistance:** After casting the vote, the voter should possess documentary proof (in the form of a receipt) to demonstrate whether he had cast a vote. But this proof should not contain any proof regards his choice of candidate.

IV. DESIGNING AN E-VOTING SYSTEM: INGREDIENTS

To include the security properties mentioned in Section III, the following components – all or some of them – may be used to design an e-voting system.

Blockchain. Blockchain is a distributed ledger that serves as a publicly accessible database, enabling interactions between multiple peer nodes. Its popularity as a technology stems from key features such as decentralization, distribution, immutability, verifiability, and transparency. In essence, it functions like a bulletin board that records transactions between parties in a permanent and tamper-resistant manner.

Construction-wise, a blockchain consists of a sequence of blocks linked together in a chain-like structure, with each block containing a collection of transactions.

- (i) **Transaction:** A transaction is the most fundamental data structure in a blockchain. It records essential information about interactions between two peer nodes, including their addresses, authentication data, the amount of cur-

rency transferred, and, most importantly, the agreements or conditions governing the exchange.

- (ii) **Block:** In the Bitcoin blockchain, multiple transactions are grouped together to form a block. Transactions within a block are organized using a Merkle tree. In this structure, transactions act as leaf nodes, and their hashes are combined recursively in pairs to form internal nodes, ultimately producing a single hash value known as the Merkle Root.

The task of block formation is assigned to specific nodes in the network, known as miners. In addition to the transaction data, each block also contains a block header, a structured data field that uniquely identifies the block. The block header includes the following components: timestamp, version field, Merkle root, difficulty target, nonce, and the hash of the previous block (see Fig. 1).

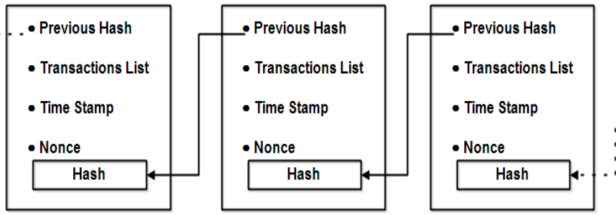


Fig. 1: Various data fields inside a block

- (iii) **Blockchain:** Once a block is formed, it is appended to the most recent block in the chain using a cryptographic hash function. Each block contains a reference to the hash of its predecessor, thereby linking all blocks back to the very first block, known as the genesis block. This sequential linking of blocks creates a chain-like structure, hence the name Blockchain [10].

Smart Contract. Earlier, we discussed agreements between two peer nodes and how such agreements can be embedded as code within transactions. These agreements are known as smart contracts.

The concept of smart contracts was first proposed by Nick Szabo in 1997. A smart contract is a self-executing program deployed on the blockchain, consisting of instructions that automatically run when predefined conditions are satisfied. These conditions are established as part of the agreement between the involved parties. By doing so, smart contracts eliminate the need for trusted third parties in defining, enforcing, and executing contractual terms.

A smart contract resides on the blockchain at a specific location, known as its contract address. Parties interact with the contract by sending transactions to this address. Since these interactions are permanently recorded on the blockchain, they are both irreversible and publicly verifiable [10]. We model the blockchain as a publicly accessible database (aka bulletin board) $\mathcal{BB}$ associated with a set of functions $\mathcal{BB}.f_1(\cdot), \mathcal{BB}.f_2(\cdot), \dots, \mathcal{BB}.f_n(\cdot)$. These functions constitute various components of the smart contract.

Public key signature. A public key signature scheme $\text{PKS} = (\text{PKS.paramGen}, \text{PKS.keyGen}, \text{PKS.sign}, \text{PKS.verify})$ is a 4-tuple of poly-time algorithms defined as follows:

- $\text{PKS.paramGen}(1^k) \rightarrow \text{param}$: On input 1^k , where k is the security parameter, this randomized algorithm generates param containing information about design parameters of the following algorithms.
- $\text{PKS.keyGen}(\text{param}) \rightarrow (Pu, Pr)$: A randomized algorithm that outputs a pair of public and private keys.
- $\text{PKS.sign}(\text{param}, m, Pr) \rightarrow \sigma$: This randomized signing algorithm takes a message m and the private key Pr as input and produces a signature σ .
- $\text{PKS.ver}(\text{param}, m, Pu, \sigma) \rightarrow \{0, 1\}$: The verification algorithm which is deterministic one that takes a message m , a signature σ , and the public key Pu as input to produce 1 (accept) if the signature is valid or 0 (reject) otherwise.

CORRECTNESS. $\text{PKS.verify}(\text{param}, m, Pu, \sigma) \rightarrow 1$, if $\text{PKS.sign}(\text{param}, m, Pr) \rightarrow \sigma$.

Various security properties of a public key signature scheme are existential unforgeability, correctness, and non-repudiation, the details of which can be found in [11].

Cryptographic hash function. A cryptographic hash function $H : D \rightarrow R$ is a deterministic algorithm that takes an arbitrary length message as input and returns a fixed-size output (here $D \in \{0, 1\}^*$ and $R \in \{0, 1\}^n$ and $n \geq 1$).

This function has several security properties, such as pre-image resistance, second pre-image resistance, and collision resistance. More details on the properties can be found in [12].

Ring signature. A ring signature scheme $\text{RS} = (\text{RS.paramGen}, \text{RS.keyGen}, \text{RS.sign}, \text{RS.ver})$ is a 4-tuple of algorithms, which are defined below:

- $\text{RS.paramGen}(1^k) \rightarrow \text{param}$ taking a security parameter k as input and outputs param containing public design information about the following algorithms.
- $\text{RS.keyGen}(\text{param}) \rightarrow (Pu, Pr)$ takes param as input and outputs a pair of public/private keys (Pu, Pr) .
- $\text{RS.sign}(\text{param}, m, \mathcal{P}, Pr) \rightarrow \sigma$ generates a signature σ on the message m using the set of public keys $\mathcal{P} = \{Pu_1, Pu_2, \dots, Pu_n\}$ and the private key of the signer Pr .
- $\text{RS.ver}(\text{param}, m, \mathcal{P}, \sigma)$ outputs a 0 or 1 for accept or reject depending on the validity of the signature σ

CORRECTNESS. $\text{RS.ver}(\text{param}, m, \mathcal{P}, \sigma) \rightarrow 1$ if and only if $\text{RS.sign}(\text{param}, m, \mathcal{P}, Pr) \rightarrow \sigma$.

Various security properties of a ring signature scheme are the signer's anonymity, unforgeability, etc., the details of which can be found in [13].

Linkable spontaneous anonymous group signature (LSAG). An LSAG scheme $\text{LSAG} = (\text{LSAG.paramGen}, \text{LSAG.keyGen}, \text{LSAG.sign}, \text{LSAG.ver}, \text{LSAG.linkVer})$ is a 5-tuple of algorithms such that:

- $\text{LSAG.paramGen}(1^k) \rightarrow \text{param}$ takes a positive integer k as input and outputs param containing public design information about the following algorithms.
- $\text{LSAG.keyGen}(\text{param}) \rightarrow (Pu, Pr)$ takes param as input and outputs a pair of public/private keys (Pu, Pr) .
- $\text{LSAG.sign}(\text{param}, m, \mathcal{P}, Pr) \rightarrow \sigma$ generates the signature σ on the message m using the parameter param , the set of public keys $\mathcal{P} = \{Pu_1, Pu_2, \dots, Pu_n\}$ and the private key of the signer Pr .
- $\text{LSAG.ver}(\text{param}, m, \mathcal{P}, \sigma)$ outputs a 0/1 for accept or reject depending on the validity of the signature σ .
- $\text{LSAG.linkVer}(\text{param}, m_1, m_2, \mathcal{P}, \sigma_1, \sigma_2)$ outputs 0/1 depending on whether the same private key has been used for generating the signatures σ_1 and σ_2 , respectively on the messages m_1 and m_2 .

CORRECTNESS OF LSAG. The correctness has two parts:

- $\text{LSAG.ver}(\text{param}, m, \mathcal{P}, \sigma) \rightarrow 1$ if and only if $\text{LSAG.sign}(\text{param}, m, \mathcal{P}, Pr) \rightarrow \sigma$.
- $\text{LSAG.linkVer}(\text{param}, m_1, m_2, \mathcal{P}, \sigma_1, \sigma_2) \rightarrow 1$, if and only if $\text{LSAG.sign}(\text{param}, m, \mathcal{P}, Pr_1) \rightarrow \sigma_1$ and $\text{LSAG.sign}(\text{param}, m, \mathcal{P}, Pr_2) \rightarrow \sigma_2$ and $Pr_1 \neq Pr_2$.

Various security properties of a LSAG signature scheme are signer's anonymity, existential unforgeability, linkability, etc., the details of which can be found in [14].

Note. For the ease of readability, the public design parameters param will be omitted from the input, for all the algorithms for the rest of the paper.

V. AN ATTACK ON THE BLOCKVOTES PROTOCOL

BlockVotes e-voting protocol was designed by Yifan Wu in 2017 [15]. Several articles have appreciated this protocol for its well-founded design supporting security and privacy properties [8], [16], [17], [18]. The protocol has also received a word of praise for its clever use of ring signature [19], [20]. In the survey presented in [21], BlockVotes was recommended for potential use in e-governance applications. In fact, according to Google Scholar, BlockVotes's design and implementation have so far been analyzed by 59 academic papers [22].

DESCRIPTION OF BLOCKVOTES E-VOTING. The original protocol, although quite innovative, suffers from lack of clarity in various parts. We first give a complete description of the original protocol by adding the missing details, and then go in for analyzing it.

The protocol uses the following cryptographic primitives as building blocks: a ring signature $\text{Ring} = (\text{ParamGen}, \text{KeyGen}, \text{Sign}, \text{Ver})$, a hash function H and a commitment scheme $\mathcal{C} = (\text{Commit}, \text{Verify})$. See Section IV to find out more on these primitives.

1. Registration Phase

- For all $i \in [n]$, the i th voter generates the public-key/private-key pair Pu_i/Pr_i , and shares Pu_i with the RA. (These keys will later be used for authentication using a ring signature.)

- The RA invokes: $\text{BB.store}(L, \{C_j\}_{j \in [m]}, \{Pu_i\}_{i \in [n]})$. Here, L : voting id; $\{C_j\}_{j \in [m]}$: set of candidates; $\{Pu_i\}_{i \in [n]}$: set of voters' public-keys.

2. Preparation Phase

- The EA generates: blockchain addresses $\{A_1, A_2, \dots, A_n, A_E\}$ and blockchain private keys $\{S_1, S_2, \dots, S_n, S_E\}$. (A_i, S_i) 's are for the voters, and (A_E, S_E) is for the EA.
- The EA sends (A_i, S_i) to a voter, denoted V_i (off-chain).

3. Voting Phase

- For all $i \in [n]$, the following steps happen.
 - The V_i selects the candidate C_j . He determines σ_i, h_i and com_i , as follows:

$$\begin{aligned}\sigma_i &:= \text{RS.Sign}(C_j, Pr_i, \{Pu_i\}_{i \in [n]}), \\ h_i &:= H(\sigma_i), \\ g_i &:= H(h_i, C_j, L).\end{aligned}$$

- The V_i invokes: $\text{BB.store}(g_i)$.

- For all $i \in [n]$, the V_i invokes: $\text{BB.store}(\sigma_i, h_i)$.

4. Tallying For all $i \in [n]$, the EA increments the vote-count of C_j by 1, if $\left(H(h_i, C_j, L) = g_i \right) \wedge \left(\text{RS.Ver}(\sigma_i, \{Pu_i\}_{i \in [n]}, C_j) = 1 \right)$.

AN ATTACK ON BLOCKVOTES E-VOTING. BlockVotes has been shown to exhibit many security properties, such as, voter anonymity, eligibility, verifiability, fairness, coercion resistance, etc. Despite, it is possible to break the anonymity property of the protocol, even in practical scenarios. The weakness arises from the strong assumption the protocol relies on. The assumption is that the RA and the EA (two powerful entities in the protocol) do not collude. We now describe what happens if they collude.

The RA is responsible for registering the voters and the candidates. So he knows the relation between the voter identity and the public key of the voter. Also, since the EA generates and provides the Bitcoin addresses to the voters, he knows the mapping between the public key and the Bitcoin address. Therefore, if the RA and the EA collude, the relation between the voter's identity and his Bitcoin address could be easily revealed to both of them. Consequently, the vote cast can be mapped to the voter itself, leading to the full break of the voter anonymity of the system.

VI. THE NEW E-VOTING PROTOCOL II

COMPONENTS. In the protocol II, three types of parties are involved: the voter $v \in \mathcal{V}$; the election authority EA; and the government GOV. The public and private keys of v are Pu_v and Pr_v . Similarly, the public and private keys of GOV are Pu_{gov} and Pr_{gov} . The GOV-issued certificate issued to v is denoted $\text{CERT}_v = \{\sigma_v, Pu_{gov}, Pu_v\}$, where $\sigma_v = \text{PKS.sign}(Pu_v, Pr_{gov})$. The symbol $\mathcal{C}$ denotes the set of candidates running for election. The L denotes the election id. We will also be using four blockchain functions as described below:

- **BB.storePub(CERT)**: Here, $CERT = \{m, Pu, \sigma\}$. This function first verifies if σ is the correct signature of m computed using the public key Pu . Then Pu is included in the set $\mathcal{P}u$. The algorithmic description is below.

BB.storePub(CERT)

if $PKS.ver(CERT) = 1$ then $\mathcal{P}u := \mathcal{P}u \cup \{Pu\}$.

- **BB.verify(σ, Pu')**: This function verifies the validity of a registration by checking the signature σ against the public key set $\mathcal{P}u$. If the signature is invalid or if any previous registration is found, the process is rejected. Upon valid registration, the tuple (σ, Pu') is added to the election record $T[k]$, and the updated index k is returned.

BB.verify(σ, Pu')

if $LSAG.ver(L, \mathcal{P}u, \sigma_v) = 1$
 if $\exists j \in \{0, 1, \dots, |T| - 1\}$,
 $LSAG.linkVer(\mathcal{P}u, L, L, \sigma_v, T[j][0]) = 1$
 abort registration.
 $k_v := \text{"Invalid Registration"}$
 else $k := |T|$
 $T[k] := (\sigma_v, Pu')$
 $k_v := k$
 else *abort registration*

- **BB.voting(σ, h, k)**: The signature σ on the hash h of the vote is verified; if valid, the hash h is stored; otherwise, the vote is marked as invalid.

BB.voting(σ, h, k)

If $PKS.ver(h, T[k][1], \sigma) = 1$,
 $T[k][2] := h$
 else *abort Vote Casting*

- **BB.tally(σ', k_v)**: This function tallies the votes by iterating over all votes in the list $T[L]$. For each vote, it verifies the integrity of the vote by checking the hash. If valid, it increments the count for the candidate c in $R[c]$.

BB.tally(c, r, k)

If $T[k][2] \neq H(c \parallel r)$, *abort Vote Casting*;
 If $c \in \mathcal{C}$: $R[c]++$;

DESCRIPTION OF PROTOCOL II. We assume the following: the $CERT_v$ has already been provided to the voter v securely; the set of candidates $\mathcal{C}$ and the election id L are already stored in the bulletin board (or blockchain) **BB**. Our protocol has been described in Fig. 2. The corresponding pictorial description is in Fig. 3.

SECURITY AND PERFORMANCE OF II. The II is secure against all attacks described in Sect. III. Most interestingly, our protocol does not use the most time-consuming LSAG primitive at the time of online vote-casting, which is dependent on the

number of voters, therefore not scalable. We move the LSAG computation to the offline registration part of the protocol. Instead our protocol uses the classical PKS primitive for online vote-casting, which does not vary with the number of voters, making it fully scalable. The key feature of our protocol is the use of both PKS and LSAG to reduce time overhead (online) as well to defend against anonymity attack. We achieve the security by combining this feature with enforcing change of blockchain addresses after registration.

VII. IMPLEMENTATION AND BENCHMARKING DETAILS

IMPLEMENTATION. Our new LSAG-based e-voting protocol (described in Sect. VI) was implemented on a private Geth-compatible blockchain network hosted at our institute's intranet. Smart contracts, written in Solidity (version $\hat{0.8.0}$), were compiled and deployed using the Hardhat development environment (v2.22.15).

The application was implemented as a full-stack `Next.js` application (v15.0.4), where both the Front- and the Back-end were written in TypeScript (v5.6.3). Additionally, the frontend was built with React (v18.2.0), styled using Tailwind CSS (v3.4.1); the backend API routes were developed on Node.js (v20.x), with blockchain interactions being handled via `ethers.js` (v6.13.4), email notifications via `nodemailer` (v6.9.16), and rate limiting via `@upstash/redis` (v1.34.3).

Version control was managed via GitHub, and the system was deployed to Vercel with an automated CI/CD pipeline triggered through a secure webhook. This setup enabled a fully integrated and reproducible environment for evaluating the complete e-voting protocol—from cryptographic operations to frontend user interaction—on the custom blockchain network.

The development was carried out on a Windows 11 operating system using Visual Studio Code as the primary text editor. The machine was equipped with an Intel Core i5 processor, which was used for executing performance benchmarks related to signing time, verification time, and signature size across varying public key counts.

The Go Language reference implementation of our LSAG scheme can be found in [23]. This LSAG scheme was invented by J. K. Liu, V. K. Wei, and D. S. Wong [14]. We first converted the Go Language reference implementation of [23] into a typescript one, since our front- and back-ends of our experimental setup were written in typescript (`cryptoUtils.ts`).

Our implementation imports relevant modules from `elliptic` (v6.6.1) (for ECC operations on the `secp256k1` curve), `bn.js` (v5.2.1) (for big number arithmetic), and `js-sha3` (v0.9.3) (for SHA3-256/512 hashing).

BENCHMARKING RESULTS. We benchmarked the key generation, signing, and verification algorithms of `cryptoUtils.ts` for various ring sizes ($n = 2$ to $n > 1000$) using a Node.js benchmarking script (`benchmarkRingSignatures.ts`) and high-resolution timers from the `perf_hooks` module. Using i5 processor/Windows11/ msi model (a part of our testbed) we

$$\Pi_{\text{LSAG,PKS},H,L,\mathcal{C},\{\text{CERT}_v\},\{Pr_v\}}(\text{BB}, \mathcal{V})$$

Input: A voter $v \in \mathcal{V}$ has LSAG and PKS schemes, a hash function H , a certificate CERT_v , and the private key Pr_v ; a bulletin board BB has the election id L and the set of candidates $\mathcal{C}$.

Output: The BB contains an array R with vote-counts for all candidates.

Step 1. [Storing identification information]

Each voter $v \in \mathcal{V}$ computes: $\text{BB.storePub}(\text{CERT}_v)$ // It computes the set of identification public keys $\mathcal{P}_u$.

(The voter v now generates another random BB address and uses it in the remainder of the protocol.)

Step 2. [Voter registration]

Each voter $v \in \mathcal{V}$ computes:

- $(Pu'_v, Pr'_v) := \text{PKS.keyGen}()$ // This is the pair of vote verification and vote casting keys.
- $\sigma_v := \text{LSAG.sign}(L, \mathcal{P}_u, Pr_v)$
- $k_v \leftarrow \text{BB.verify}(\sigma_v, Pu'_v)$ // This generates the list of eligible voters T .

Step 3. [Vote casting]

Every voter $v \in \mathcal{V}$ computes:

- The voter v selects the candidate $c \in \mathcal{C}$
- The voter chooses a random number r
- $h_v := H(c \parallel r)$ // Here the voter hides and commits to her vote.
- $\sigma'_v := \text{PKS.sign}(h_v, Pr'_v)$
- $\text{BB.voting}(\sigma'_v, h_v, k_v)$

(The next step starts after all voters cast their votes.)

Step 4. [Vote Tallying] Every voter $v \in \mathcal{V}$ computes: $\text{BB.tally}(c, r, k_v)$ // Here the voter reveals her vote.

Fig. 2: Algorithmic description of Π (for pictorial description see Fig. 3)

benchmarked key generation, key image generation (essential for linkability), signing, verification, and key image lookup (for detecting key image reuse) of our LSAG reference implementation scheme. These operations were timed across different ring sizes n (the number of public keys in the ring) to observe how their runtimes scale. Results are measured in milliseconds (ms), and the signature size in bytes is also recorded to demonstrate its growth relative to n .

The KeyGen, Sign, and Verify operations exhibit a clear linear growth trend with increasing n , validating their theoretical $O(n)$ complexity. This is expected, as key generation involves creating n elliptic curve key pairs, while signing and verification necessitate iterating over all n ring members, performing several scalar multiplications and hash computations per member. Verification times are comparable to signing times. Key Image Generation also scales with n , primarily due to hashing the n public keys, though its initial growth relative to n appears slower; this is likely because fixed-cost operations within this step are more dominant at smaller n , with the public key hashing cost becoming more prominent for

TABLE I: LSAG Performance and Signature Size Benchmarking Using i5 processor/Windows11/ msi model

Ring Size (n)	Key Gen (ms)	Sign Time (ms)	Verify Time (ms)	Signature Size (bytes)
2	27.50	22.05	43.77	160
10	8.00	57.09	46.81	416
50	30.27	236.69	225.19	1696
100	67.56	474.59	453.79	3296
200	126.78	931.83	909.75	6496
300	173.60	1428.55	1325.30	9696
400	231.20	1827.63	1777.60	12 896
500	283.86	2274.83	2272.10	16 096
600	335.08	2725.47	2692.28	19 296
700	393.55	3167.21	3086.56	22 496
800	447.48	3623.25	3569.69	25 696
900	506.22	4090.17	4082.17	28 896
1000	552.90	4569.52	4472.98	32 096

larger rings. Finally, Key Image Lookup time remains nearly constant and highly efficient, confirming the effectiveness of set-based lookups for managing key images.

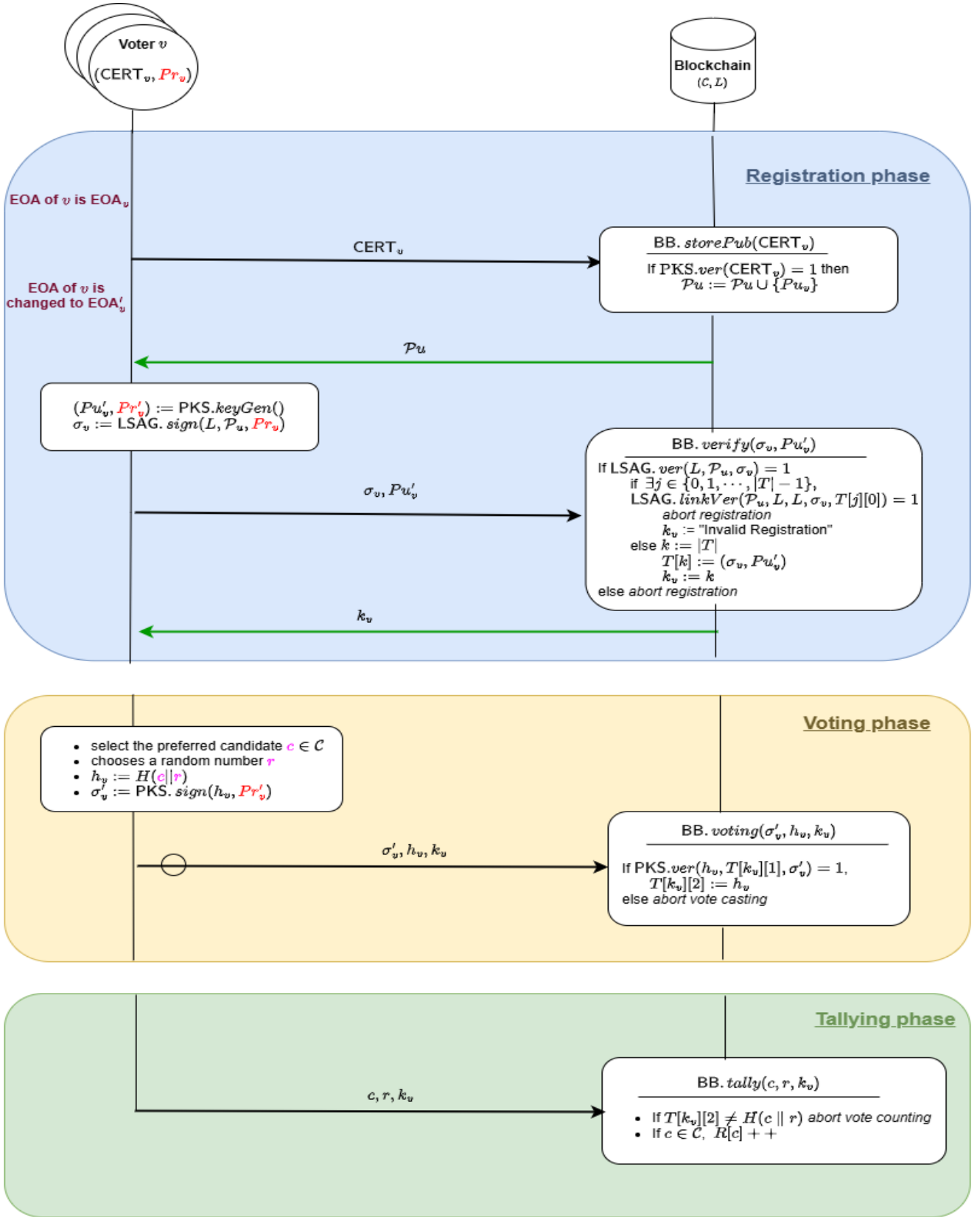


Fig. 3: The new e-voting protocol II. Here $CERT_v = \{\sigma_v, Pu_{gov}, Pu_v\}$ where $\sigma_v = PKS.sign(Pu_v, Pr_{gov})$. The Pu_{gov} , and Pr_{gov} are the public and private keys of the government which are generated as $(Pu_{gov}, Pr_{gov}) \leftarrow PKS.keyGen()$. The Pu_v and Pr_v are the public and private keys of voter which are generated as follows $(Pu_v, Pr_v) \leftarrow LSAG.keyGen()$. C is the set of candidates and L is the election id. T and R are two arrays containing voting and counting data. The circle symbol at the tail of an arrow indicates all voters v need to complete the step before the next step begins. After storing $CERT_v$ voter uses a different blockchain address.

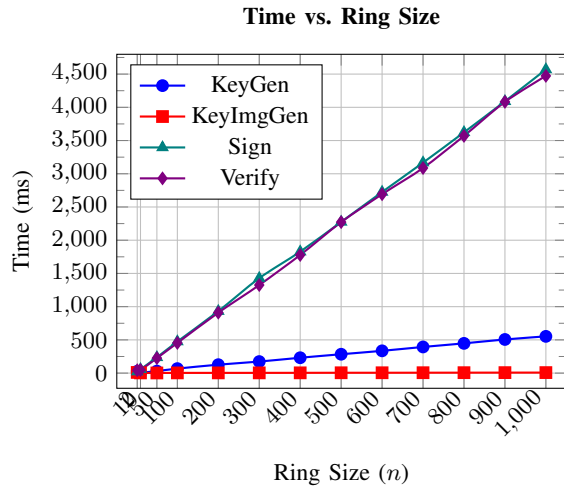


Fig. 4: Graphical representation of Table I; Relationship between ring size and time (ms)

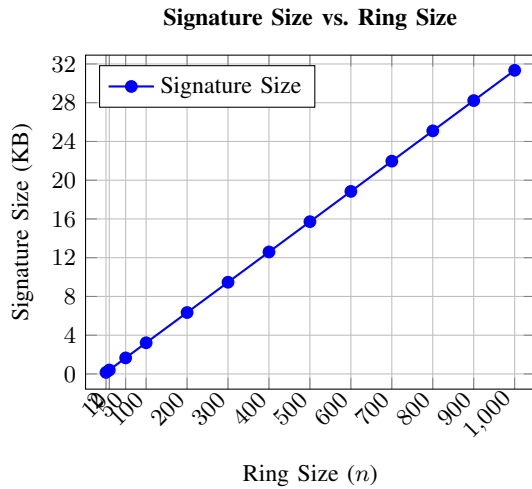


Fig. 5: Graphical representation of Table I; Relationship between ring size and signature size

VIII. CONCLUSION

In this paper, we break the anonymity property of a highly popular e-voting scheme BLOCKVOTES. We also design a novel and secure e-voting system based on a ring signature. More interestingly, unlike all other existing ring signature based e-voting schemes, the online phase (the actual vote-casting phase) of our protocol is extremely efficient and fully scalable. This is achieved due to clever applications of both public key and the traditional signatures in online and offline phases. We implemented our e-voting scheme in our native institutional blockchain testbed and carried out benchmarking experiments. The performance numbers show the practicality of the protocol for real-time use.

REFERENCES

[1] F. S. Hardwick, A. Gioulis, R. N. Akram, and K. Markantonakis, “E-voting with blockchain: An e-voting protocol with decentralisation and

voter privacy,” in *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*. IEEE, 2018, pp. 1561–1567.

[2] P. Mccorry, M. Mehrnezhad, E. Toreini, S. F. Shahandashti, and F. Hao, “On secure e-voting over blockchain,” *Digital Threats: Research and Practice (DTRAP)*, vol. 2, no. 4, pp. 1–13, 2021.

[3] G. G. Dagher, P. B. Marella, M. Milojkovic, and J. Mohler, “Broncovote: Secure voting system using ethereum’s blockchain,” 2018.

[4] K. Lee, J. I. James, T. G. Ejeta, and H. J. Kim, “Electronic voting service using block-chain,” *Journal of Digital Forensics, Security and Law*, vol. 11, no. 2, p. 8, 2016.

[5] A. B. Ayed, “A conceptual secure blockchain-based electronic voting system,” *International Journal of Network Security & Its Applications*, vol. 9, no. 3, pp. 01–09, 2017.

[6] F. . Hjalmarsson, G. K. Hreifarsson, M. Hamdaqa, and G. Hjalmtýsson, “Blockchain-based e-voting system,” in *2018 IEEE 11th international conference on cloud computing (CLOUD)*. IEEE, 2018, pp. 983–986.

[7] K.-H. Wang, S. K. Mondal, K. Chan, and X. Xie, “A review of contemporary e-voting: Requirements, technology, systems and usability,” *Data Science and Pattern Recognition*, vol. 1, no. 1, pp. 31–47, 2017.

[8] F. Rabia, A. Sara, and T. Gadi, “A survey on e-voting based on blockchain,” in *Proceedings of the 4th International Conference on Networking, Information Systems & Security*, 2021, pp. 1–8.

[9] I. Stančiková and I. Homoliak, “Sbvote: Scalable self-tallying blockchain-based voting,” *arXiv preprint arXiv:2206.06019*, 2022, cryptography and Security (cs.CR); Distributed, Parallel, and Cluster Computing (cs.DC). [Online]. Available: <https://arxiv.org/abs/2206.06019>

[10] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” <https://bitcoin.org/bitcoin.pdf>, 2008.

[11] S. Goldwasser, S. Micali, and R. L. Rivest, “A digital signature scheme secure against adaptive chosen-message attacks,” *SIAM Journal on Computing*, vol. 17, no. 2, pp. 281–308, 1988. [Online]. Available: <https://doi.org/10.1137/0217017>

[12] W. Macharia, “Cryptographic hash functions,” *mai. de*, 2021.

[13] A. Bender, J. Katz, and R. Morselli, “Ring signatures: Stronger definitions, and constructions without random oracles,” *Journal of Cryptology*, vol. 22, no. 1, pp. 114–138, 2009.

[14] J. K. Liu, V. K. Wei, and D. S. Wong, “Linkable spontaneous anonymous group signature for ad hoc groups,” in *ACISP*, vol. 4. Springer, 2004, pp. 325–335.

[15] Y. Wu, “An e-voting system based on blockchain and ring signature,” *Master. University of Birmingham*, 2017.

[16] A. Razzaq, M. M. Khan, R. Talib, A. D. Butt, N. Hanif, S. Afzal, and M. R. Raouf, “Use of blockchain in governance: A systematic literature review,” *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 5, 2019.

[17] S. Gupta, A. Gupta, I. Y. Pandya, A. Bhatt, and K. Mehta, “End to end secure e-voting using blockchain & quantum key distribution,” *Materials Today: Proceedings*, vol. 80, pp. 3363–3370, 2023.

[18] R. Chhabra, U. Vohra, V. Khanna, A. Verman, P. Tanwar, and B. Kumar, “The next gen election: Design and development of e-voting web application,” in *2020 5th International Conference on Communication and Electronics Systems (ICCES)*. IEEE, 2020, pp. 536–541.

[19] A. Benabdallah, A. Audras, L. Coudert, N. El Madhoun, and M. Badra, “Analysis of blockchain solutions for e-voting: A systematic literature review,” *IEEE Access*, vol. 10, pp. 70 746–70 759, 2022.

[20] W. Ni, P. Cheng, L. Chen, and X. Lin, “When the recursive diversity anonymity meets the ring signature,” in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 1359–1371. [Online]. Available: <https://doi.org/10.1145/3448016.3452825>

[21] J. Bernal Bernabe, J. L. Canovas, J. L. Hernandez-Ramos, R. Torres Moreno, and A. Skarmeta, “Privacy-preserving solutions for blockchain: Review and challenges,” *IEEE Access*, vol. 7, pp. 164 908–164 940, 2019.

[22] https://scholar.google.com/scholar?hl=en&as_sdt=0%2C5&q=An+e-voting+system+based+on+blockchain+and+ring+signature&btnG=

[23] “Go language implementation of LSAG invented by J. K. Liu, V. K. Wei, and D. S. Wong (ACISP),” <https://github.com/zbohm/lirisi.git>, 2021.